

## Branch 1 — Synthetic Voice Detection Using AASIST

### Project

### AI-Powered Real-Time Detection and Prevention of Voice Cloning Impersonation Attacks

### Module

### Branch 1 — Synthetic Voice Detection

### Model

### AASIST — Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks

### Status

### Proof of Concept (POC) — Completed

---

### Recommended Usage by the Main Project

The main project should primarily consume:

```
synthetic_probability = result["synthetic_probability"]
```

For example:

```
from synthetic_detection import analyze_synthetic_voice
```

```
result = analyze_synthetic_voice("audio.wav")
```

```
synthetic_score = result["synthetic_probability"]
```

```
print("Synthetic score:", synthetic_score)
```

This score can later be passed to the project's **score-fusion / risk-analysis layer**.

Conceptually:

Audio

↓

AASIST Module

↓

synthetic\_probability

↓

Score Fusion

↓

Risk Engine

The AASIST module itself should **not** be treated as the final fraud/security decision engine.

---

### What the Developer Does NOT Need to Handle

A developer using `analyze_synthetic_voice()` does not need to manually:

- Load the AASIST model
- Load AASIST.pth
- Load AASIST.conf
- Convert stereo to mono
- Resample audio to 16 kHz
- Prepare the 64,600-sample input
- Run PyTorch inference
- Apply softmax

These operations are already handled internally by:

`analyze_synthetic_voice()`

Therefore, the intended developer workflow is simply:

```
from synthetic_detection import analyze_synthetic_voice
```

```
result = analyze_synthetic_voice("audio.wav")
```

and then:

```
synthetic_score = result["synthetic_probability"]
```

---

### Important Input Requirement

The module accepts an audio file path.

Internally, the module handles:

Different sample rates → Resampled to 16 kHz

Stereo audio → Converted to mono

Short audio → Adjusted to required model length

Long audio → Currently first ~4.04 seconds analyzed

The AASIST model operates on approximately:

16 kHz × 64,600 samples ≈ 4.04 seconds

For long recordings, complete recording-level analysis and score aggregation still require further validation.

---

### Minimal Integration Example

For another team member, the required code is only:

```
from synthetic_detection import analyze_synthetic_voice
```

```
result = analyze_synthetic_voice("audio.wav")
```

```
synthetic_score = result["synthetic_probability"]
```

```
print(synthetic_score)
```

Example:

0.86

This `synthetic_score` is the primary value that can be consumed by the next stage of the SIH architecture.

## 1. Objective

The objective of Branch 1 is to develop and validate a synthetic/spoofed voice detection component using a pretrained AASIST model.

The component should accept an audio file and produce a synthetic/spoof score that can later be consumed by the project's Feature/Score Fusion and Risk Engine components.

The target interface is:

```
from synthetic_detection import analyze_synthetic_voice
```

```
result = analyze_synthetic_voice("audio.wav")
```

Example output:

```
{  
  "synthetic_probability": 0.9997,  
  "bonafide_probability": 0.0003,  
  "inference_time_seconds": 0.3877,  
  "result": "SYNTHETIC / SPOOF"  
}
```

The current implementation is intended as a **working POC**, not a production-grade fraud decision system.

---

## 2. Model Selected

The selected model for Branch 1 is **AASIST**.

AASIST is an audio anti-spoofing model designed to distinguish between:

- **Bonafide speech** — genuine human speech
- **Spoof speech** — artificially generated or manipulated speech

The model uses spectro-temporal representations and graph attention mechanisms to learn characteristics that distinguish genuine and spoofed speech.

For this POC, the pretrained AASIST checkpoint was used rather than training the model from scratch.

---

## 3. Implementation Used

The implementation used for the POC is the official AASIST repository:

`clovaai/aasist`

The project was cloned locally and integrated into the custom detection module.

The relevant model implementation is:

`aasist/models/AASIST.py`

The configuration used is:

`aasist/config/AASIST.conf`

The pretrained checkpoint used is:

aasist/models/weights/AASIST.pth

### **Model configuration**

The configuration specifies:

Architecture: AASIST

Input samples: 64600

First convolution: 128

Filter configuration: [70, [1,32], [32,32], [32,64], [64,64]]

GAT dimensions: [64,32]

---

## **4. Development Environment**

### **Operating System**

Windows

### **Project Directory**

D:\M.Tech\VOICE\_CLONING\_DETECTOR

### **Python Environment**

A dedicated Conda environment was created:

voice\_clone

Python version:

Python 3.10.21

### **PyTorch**

PyTorch 2.11.0+cpu

### **Torchaudio**

Torchaudio 2.11.0+cpu

### **CUDA**

CUDA available: False

Therefore, all experiments were performed using CPU inference.

### **Hardware**

The tested system does not have an NVIDIA CUDA-capable GPU.

The available graphics hardware was Intel Iris Xe.

---

## 5. Project Structure

The relevant POC structure is:

VOICE\_CLONING\_DETECTOR/

```
|
|— synthetic_detection.py
|— test_synthetic.py
|— benchmark.py
|— test_windows.py
|— window_aggregation.py
|— AASIST_NOTES.txt
|
└─ aasist/
    |
    |— config/
    |   └─ AASIST.conf
    |
    |— models/
    |   |— AASIST.py
    |   └─ weights/
    |       |— AASIST.pth
    |       └─ AASIST-L.pth
    |
    |— data_utils.py
    |— evaluation.py
    |— main.py
    └─ requirements.txt
```

The Branch-1 POC uses:

AASIST.pth

The AASIST-L.pth checkpoint was not used.

RawNet2 was also **not used**, as it is outside the current Branch-1 scope.

---

## 6. Audio Processing Pipeline

The implemented pipeline is:

Audio File

|



Load Audio

|



Stereo → Mono

|



Convert to Float32

|



Resample to 16 kHz

|



Prepare 64,600 Samples

|



AASIST Model

|

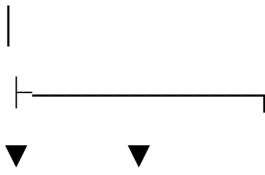


Two-Class Output

|



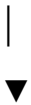
Softmax



Spoof Score    Bonafide Score



Synthetic Probability



POC Decision

---

## 7. Audio Input Requirements

The current implementation prepares audio according to the AASIST model configuration.

### Sampling Rate

Target:

16,000 Hz

If the input has another sample rate, it is resampled to 16 kHz.

For example, a 48 kHz recording was successfully processed.

### Channels

The model input is converted to mono.

For stereo audio:

Left + Right



Average



Mono

A stereo-input test was successfully completed.

### Model Input Length

The configured input size is:

64,600 samples

At 16 kHz:

64,600 / 16,000

≈ 4.04 seconds

Therefore, the current POC effectively processes approximately **4.04 seconds of audio per inference window**.

---

## 8. Short Audio Handling

If the input audio contains fewer than 64,600 samples, the current implementation repeats the audio until the required number of samples is reached.

Conceptually:

Short Audio

↓

Repeat

↓

64,600 samples

↓

AASIST

This was implemented to satisfy the model's fixed input requirement.

### Limitation

Repeating a short recording can introduce artificial repetition into the model input.

Therefore, this approach should be considered a **POC preprocessing strategy**, not necessarily the optimal production strategy.

A future implementation should evaluate alternatives such as padding or a more carefully designed short-segment handling strategy.

---

## 9. Long Audio Handling

The current `analyze_synthetic_voice()` function processes only the first approximately 4.04 seconds of a longer recording.

For example:

60-second audio

↓

First ~4.04 seconds

↓

AASIST

It does not currently analyze the complete recording.

A window-based experiment was therefore performed to understand how AASIST behaves across different portions of a long recording.

---

## 10. Model Output Interpretation

The AASIST model produces two output values.

Based on the implementation and the official evaluation logic:

Class 0 = Spoof

Class 1 = Bonafide

The model output is converted using softmax:

```
probabilities = torch.softmax(output, dim=1)
```

Then:

```
spoof_probability = probabilities[0][0]
```

```
bonafide_probability = probabilities[0][1]
```

The backend-facing synthetic score is:

```
synthetic_probability = spoof_probability
```

Therefore:

```
synthetic_probability
```

=

```
softmax-derived AASIST spoof score
```

---

## 11. Important Terminology

The value returned as:

```
synthetic_probability
```

should currently be understood as a **softmax-derived synthetic/spoof score**.

It should not yet be interpreted as a scientifically calibrated probability.

For example:

synthetic\_probability = 0.9997

means that the model's softmax output strongly favors the spoof class.

It does **not** by itself establish that there is a calibrated real-world 99.97% probability that the recording is synthetic.

Probability calibration and validation against a suitable dataset would be required before making such a claim.

---

## 12. POC Decision Threshold

The current POC uses:

Threshold = 0.5

Decision logic:

Synthetic score  $\geq 0.5$

↓

SYNTHETIC / SPOOF

Synthetic score  $< 0.5$

↓

BONAFIDE / REAL

This threshold is an **initial operational POC threshold**.

It has not been statistically optimized or validated on a sufficiently large representative dataset.

The final production threshold should be selected after validation using appropriate genuine and spoof datasets and application-specific false-positive/false-negative requirements.

---

## 13. Real and Synthetic Speech Testing

Multiple real and synthetic audio tests were performed.

### Basic Test Results

| Test                  | Synthetic Score | Bonafide Score | Result    |
|-----------------------|-----------------|----------------|-----------|
| Real audio            | 0.0001          | 0.9999         | BONAFIDE  |
| Synthetic audio       | 0.7552          | 0.2448         | SYNTHETIC |
| Real short audio      | 0.0001          | 0.9999         | BONAFIDE  |
| Synthetic short audio | 0.7552          | 0.2448         | SYNTHETIC |
| Real 48 kHz audio     | 0.0000          | 1.0000         | BONAFIDE  |
| Real stereo audio     | 0.0001          | 0.9999         | BONAFIDE  |
| Real WhatsApp MPEG    | 0.0115          | 0.9885         | BONAFIDE  |

These tests demonstrate that the current preprocessing and inference pipeline can process different input conditions.

---

#### 14. Latest Validation Test

After cleaning and adding empty-audio validation, a synthetic recording was tested again.

Result:

Synthetic probability : 0.9997

Bonafide probability : 0.0003

Inference time : 0.3877 seconds

Result : SYNTHETIC / SPOOF

This confirms that the cleaned implementation continues to produce the expected spoof classification.

---

#### 15. Different Sample Rate Testing

A real recording with a sample rate of 48 kHz was tested.

Result:

Synthetic probability : 0.0000

Bonafide probability : 1.0000

Result : BONAFIDE / REAL

This demonstrates that the preprocessing pipeline successfully performs sample-rate conversion to the model's target 16 kHz format.

---

## 16. Stereo Audio Testing

A stereo audio test was also performed.

The stereo input was converted to mono before inference.

Result:

Synthetic probability : 0.0001

Bonafide probability : 0.9999

Result : BONAFAIDE / REAL

### Note

The stereo test file was created by duplicating the same mono signal into both channels.

Therefore, this test validates the **stereo-to-mono preprocessing path**, but it does not represent a naturally recorded stereo signal with different left/right channels.

---

## 17. WhatsApp Audio Testing

A real WhatsApp audio recording was tested directly.

Result:

Synthetic probability : 0.0115

Bonafide probability : 0.9885

Result : BONAFAIDE / REAL

This demonstrates that the current audio-loading pipeline was able to process the tested WhatsApp MPEG recording.

However, compatibility with every possible WhatsApp audio container/codecs should not be assumed without additional testing.

---

## 18. Long Real Recording Experiment

A real recording of approximately:

56.35 seconds

was analyzed using multiple ~4-second windows.

Observed scores:

| Window | Synthetic Score |
|--------|-----------------|
|--------|-----------------|

|           |        |
|-----------|--------|
| 00–04 sec | 0.9995 |
|-----------|--------|

|           |        |
|-----------|--------|
| 10–14 sec | 0.9562 |
|-----------|--------|

|           |        |
|-----------|--------|
| 20–24 sec | 0.0059 |
|-----------|--------|

|           |        |
|-----------|--------|
| 30–34 sec | 0.7075 |
|-----------|--------|

|           |        |
|-----------|--------|
| 40–44 sec | 0.9994 |
|-----------|--------|

|           |        |
|-----------|--------|
| 50–54 sec | 0.1719 |
|-----------|--------|

This experiment produced an important observation:

A genuine long recording can produce highly variable AASIST spoof scores across different audio segments.

For example, some windows from the genuine recording produced scores close to 1.0, while another window produced a score close to 0.006.

Therefore, a single maximum window score should not automatically be used as the synthetic score for an entire recording.

---

## 19. Long Synthetic Recording Experiment

A synthetic recording of approximately:

60 seconds

was also tested across multiple windows.

Results:

| Window | Synthetic Score |
|--------|-----------------|
|--------|-----------------|

|           |        |
|-----------|--------|
| 00–04 sec | 0.9994 |
|-----------|--------|

|           |        |
|-----------|--------|
| 10–14 sec | 1.0000 |
|-----------|--------|

|           |        |
|-----------|--------|
| 20–24 sec | 0.9981 |
|-----------|--------|

|           |        |
|-----------|--------|
| 30–34 sec | 0.9999 |
|-----------|--------|

|           |        |
|-----------|--------|
| 40–44 sec | 1.0000 |
|-----------|--------|

|           |        |
|-----------|--------|
| 50–54 sec | 0.9998 |
|-----------|--------|

The synthetic recording produced consistently high spoof scores across the tested windows.

This provides useful POC evidence that the tested synthetic recording was strongly detected by AASIST.

However, this experiment involved only one real and one synthetic long recording and therefore cannot be used to calculate general detection accuracy.

---

## 20. Window Score Aggregation Experiment

The observed long-recording scores were summarized using:

- Mean
- Median
- Maximum

### Real Recording

Mean score : 0.6401

Median score : 0.8318

Maximum score: 0.9995

### Synthetic Recording

Mean score : 0.9995

Median score : 0.9998

Maximum score: 1.0000

This experiment suggests that aggregation strategy is an important design consideration for long recordings.

The maximum score is particularly problematic because the genuine recording also produced a maximum score of 0.9995.

Therefore:

**Maximum-window score should not be selected as the final long-recording synthetic score without substantial validation.**

Mean or median aggregation may be worth investigating, but the correct strategy must be established using a representative validation dataset.

---

## 21. Inference Latency Benchmark

A dedicated benchmark was performed after three warm-up runs.

Ten inference runs produced:

Run 1 : 0.3045 s

Run 2 : 0.2929 s

Run 3 : 0.3275 s

Run 4 : 0.3402 s

Run 5 : 0.3012 s

Run 6 : 0.2949 s

Run 7 : 0.3046 s

Run 8 : 0.3032 s

Run 9 : 0.3000 s

Run 10 : 0.3057 s

Summary:

Average inference time : 0.3075 seconds

Minimum : 0.2929 seconds

Maximum : 0.3402 seconds

Therefore, the observed model inference latency is approximately:

**0.31 seconds per ~4.04-second audio window on the tested CPU system.**

### **Important**

This benchmark measures **model inference**, not complete end-to-end application latency.

It does not include:

- Network transmission
  - Android processing
  - FastAPI request handling
  - File upload time
  - Disk I/O
  - Audio decoding overhead
  - Risk Engine processing
  - Feature fusion
-

## 22. Current POC Interface

The module provides a simple interface:

```
from synthetic_detection import analyze_synthetic_voice
```

```
result = analyze_synthetic_voice(  
    "audio.wav"  
)
```

The returned structure is:

```
{  
    "synthetic_probability": 0.9997,  
    "bonafide_probability": 0.0003,  
    "inference_time_seconds": 0.3877,  
    "result": "SYNTHETIC / SPOOF"  
}
```

This makes the module suitable for later integration with the project's backend components.

---

## 23. Proposed Future Integration

The AASIST module should eventually be consumed by the larger system as one signal.

The overall project architecture is:

Audio

↓

Preprocessing / VAD

↓

Multi-model AI Analysis

↓

Feature / Score Fusion

↓

Risk Engine

↓

Security Policy



FastAPI



Android

Branch 1 contributes:

Audio



AASIST



Synthetic / Spoof Score



Feature / Score Fusion

The AASIST score should therefore be treated as **one evidence signal**, rather than the final fraud decision.

---

## 24. Recommended Future Backend Interface

The current interface can later be integrated approximately as:

```
result = analyze_synthetic_voice(audio_file)
```

```
synthetic_score = result["synthetic_probability"]
```

The score can then be passed to the project's Feature/Score Fusion layer.

For example:

AASIST synthetic score



Feature/Score Fusion



Risk Engine

The Risk Engine should ultimately combine this signal with the other security signals defined by the project architecture.

No Risk Engine or FastAPI implementation was included in this Branch-1 POC.

---

## **25. Limitations Identified**

The following limitations were identified during development and testing.

### **25.1 Limited validation dataset**

The current tests demonstrate functionality but do not constitute a statistically valid evaluation.

A larger dataset containing diverse:

- Genuine speakers
- Synthetic voices
- Voice-cloned voices
- Different recording devices
- Different microphones
- Different environments
- Different codecs
- Different languages
- Different noise levels

would be required for meaningful performance evaluation.

---

### **25.2 Single-window processing**

The current main function analyzes approximately 4.04 seconds of audio.

Long recordings are not automatically processed end-to-end.

---

### **25.3 Long-recording score variation**

Different segments of the same genuine recording can produce substantially different scores.

Therefore, long-recording aggregation requires further investigation.

---

#### **25.4 Short-audio repetition**

Short recordings are currently repeated to satisfy the fixed input length.

This is a practical POC solution but may not be optimal for production.

---

#### **25.5 Threshold not validated**

The current:

0.5

threshold is only an initial POC threshold.

It should not be considered a final production threshold.

---

#### **25.6 Softmax score is not calibrated**

The synthetic score is derived from the model's softmax output.

Calibration should be performed before interpreting the score as a true probability.

---

#### **25.7 CPU-only inference**

Testing was performed without an NVIDIA GPU.

Production latency may differ significantly on other hardware.

---

### **26. Problems Encountered During Development**

The following technical issues were encountered/resolved during setup and experimentation:

#### **Python/environment compatibility**

The system Python version was unsuitable for the desired setup, so a dedicated Conda environment was created using Python 3.10.

Final environment:

Python 3.10.21

#### **PyTorch configuration**

The environment uses:

PyTorch 2.11.0+cpu

Torchaudio 2.11.0+cpu

with:

CUDA = False

### **Model integration**

The official AASIST model implementation and pretrained checkpoint were successfully loaded locally.

### **Audio preprocessing**

The preprocessing pipeline was implemented to handle:

- Sample-rate conversion
- Stereo-to-mono conversion
- Fixed model input length
- Short audio
- Multiple audio formats

### **Empty audio**

Additional validation was added to prevent empty audio files from reaching the fixed-length processing stage.

---

## **27. What Has Been Successfully Completed**

The Branch-1 POC has successfully demonstrated:

- AASIST implementation identified
- Pretrained checkpoint obtained
- Dedicated Python environment created
- AASIST dependencies installed
- AASIST model loaded locally
- CPU inference successfully executed
- Genuine speech tested
- Synthetic speech tested
- Short audio tested
- 48 kHz audio tested
- Stereo audio tested
- WhatsApp MPEG audio tested

- Long recording behavior investigated
  - Window-based analysis investigated
  - Score aggregation investigated
  - Inference latency benchmarked
  - Clean Python inference function created
  - Input/output behavior documented
  - Limitations identified
- 

## **28. Current Status**

### **Branch 1 — Synthetic Voice Detection**

#### **Status: POC COMPLETE**

The current system successfully provides:

Audio File

↓

Preprocessing

↓

Pretrained AASIST

↓

Spoof/Synthetic Score

↓

Result

The primary reusable function is:

```
analyze_synthetic_voice(audio_file)
```

The current POC demonstrates that AASIST can distinguish the tested genuine and synthetic recordings and provides a usable spoof score for later system-level integration.

---

## How Other Team Members Can Use the AASIST Module

The AASIST synthetic voice detection module is designed as a simple Python function so that other components of the project do not need to know the internal AASIST implementation.

### 1. Import the function

```
from synthetic_detection import analyze_synthetic_voice
```

### 2. Pass an audio file

```
result = analyze_synthetic_voice("audio.wav")
```

Replace "audio.wav" with the path of the audio file that needs to be analyzed.

For example:

```
result = analyze_synthetic_voice(  
    r"D:\path\to\your\audio.wav"  
)
```

### 3. Print the result

```
print(result)
```

### Complete Example

```
from synthetic_detection import analyze_synthetic_voice
```

```
result = analyze_synthetic_voice("audio.wav")
```

```
print(result)
```

### Example Output

```
{  
    "synthetic_probability": 0.7552,  
    "bonafide_probability": 0.2448,  
    "inference_time_seconds": 0.4481,  
    "result": "SYNTHETIC / SPOOF"  
}
```

---

## Understanding the Output

The function returns a Python dictionary containing four values:

### **synthetic\_probability**

```
result["synthetic_probability"]
```

This is the AASIST-derived **spoof/synthetic score**.

Example:

0.7552

Higher values indicate that the model considers the audio more likely to be spoof/synthetic.

**Important:** This is currently a softmax-derived score and is not a calibrated real-world probability.

---

### **bonafide\_probability**

```
result["bonafide_probability"]
```

This is the model's score for genuine/bonafide speech.

Example:

0.2448

The two scores approximately sum to 1:

$\text{synthetic\_probability} + \text{bonafide\_probability} \approx 1$

---

### **inference\_time\_seconds**

```
result["inference_time_seconds"]
```

This shows how long the AASIST model took to perform inference.

Example:

0.4481 seconds

The measured CPU inference benchmark for one approximately 4.04-second model window is around **0.31 seconds** on the development machine.

---

### **result**

```
result["result"]
```

This is the current POC classification:

SYNTHETIC / SPOOF

or

BONAFIDE / REAL

The current POC uses:

`synthetic_probability >= 0.5`

as the synthetic/spoof threshold.

**Note:** The 0.5 threshold is an initial POC threshold and has not yet been validated on a representative dataset.

---